

Chess Bot v1

Un Transformer qui joue aux echecs sans recherche

Projet de substitution au stage

SEIBOU Naadjath & LAKRA Rajaa

ECE, soutenance semaine du 31 aout 2026

La question de depart

- ▶ **Les moteurs classiques CHERCHENT.**

Deep Blue, Stockfish : ils explorent des millions de positions futures avant de jouer un coup.

- ▶ **Notre bot, lui, ne cherche jamais.**

Il repond a l'instinct, comme un joueur humain fort en partie a la pendule tres rapide (blitz).

- ▶ **La question qu'on se pose :**

un Transformer peut-il jouer correctement rien qu'en RECONNAISSANT une position, sans jamais calculer une suite de coups ?

Contexte : les echecs et l'IA

Un demi-siecle d'approches, un point commun : la recherche

- ▶ **1997 : Deep Blue bat Kasparov**

recherche alpha-beta massive, evaluation ecrite a la main par des experts

- ▶ **2017 : AlphaZero**

reseau de neurones + recherche arborescente (MCTS), entraine par auto-apprentissage

- ▶ **2020+ : Stockfish (NNUE)**

recherche alpha-beta tres optimisee + petit reseau d'evaluation

- ▶ **2024 : DeepMind, Grandmaster-Level Chess Without Search**

un Transformer SEUL atteint un niveau de grand maitre, c'est notre reference directe

Notre pari : une version reduite et assumee

- ▶ **DeepMind (2024)**

270 millions de parametres · 10 millions de parties annotees par Stockfish · TPU dedies

- ▶ **Nous**

6,86 millions de parametres (x40 plus petit) · 1 million de positions · 1 GPU T4 gratuit (Google Colab)

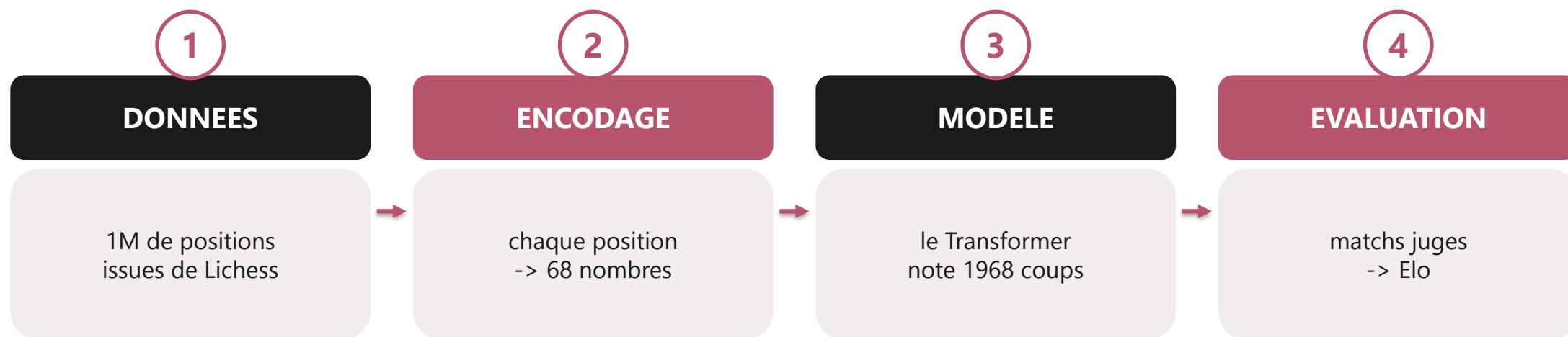
- ▶ **Consequence assumee**

on ne vise pas a battre Stockfish. L'objectif est de mesurer honnetement ce qu'un petit modele apprend, avec la meme rigueur experimentale.

COMMENT CA MARCHE

Vue d'ensemble

Quatre etapes, de la partie brute au coup joué



L'entrainement est du clonage comportemental : le reseau apprend a imiter le coup joue par un joueur fort dans chaque position.

DONNEES

Etape 1 : les donnees

Filtrer pour apprendre d'un bon joueur, pas de n'importe qui

186 057 parties lues

archive Lichess de janvier 2024, lue en flux (aucune decompression sur le disque)

Filtres appliques

Elo des deux joueurs ≥ 2000 · cadence ≥ 180 s (exclut le bullet) · partie terminee · 8 premiers demi-coups ignores (ouverture memorisee)

14 456 parties gardees

soit 7,8 % des parties lues

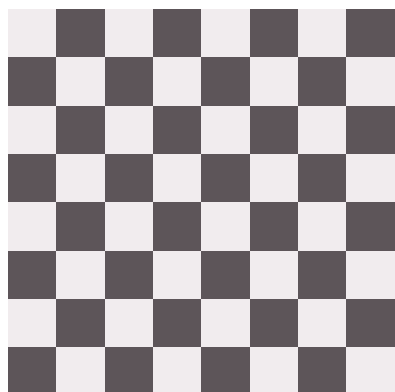
1 000 012 positions

980 440 pour l'entrainement · 19 572 pour la validation (separees PAR PARTIE, jamais par position, pour eviter la fuite de donnees)

ENCODAGE

Etape 2 : encoder une position

Une position d'echecs devient une sequence de 68 nombres



L'echiquier : 64 cases



tokens 0 - 63

contenu de chaque case (0 = vide, 1-6 = pieces allies, 7-12 = pieces adverses)

token 64

le trait (qui doit jouer)

tokens 65 - 66

droits de roque

token 67

case de prise en passant

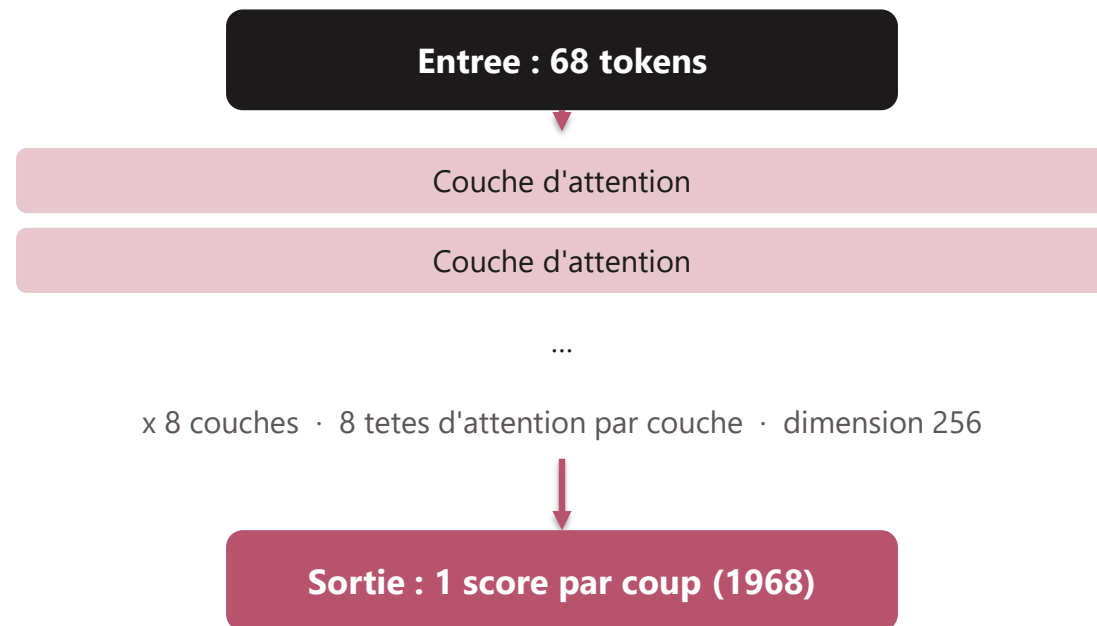
= 68 tokens en entree du reseau

Astuce cle : l'echiquier est toujours retourne du point de vue du joueur au trait. Le modele n'apprend ainsi qu'un seul point de vue, deux fois plus vite.

ARCHITECTURE

Etape 3 : le modele

Un Transformer encodeur de 6,86 millions de parametres



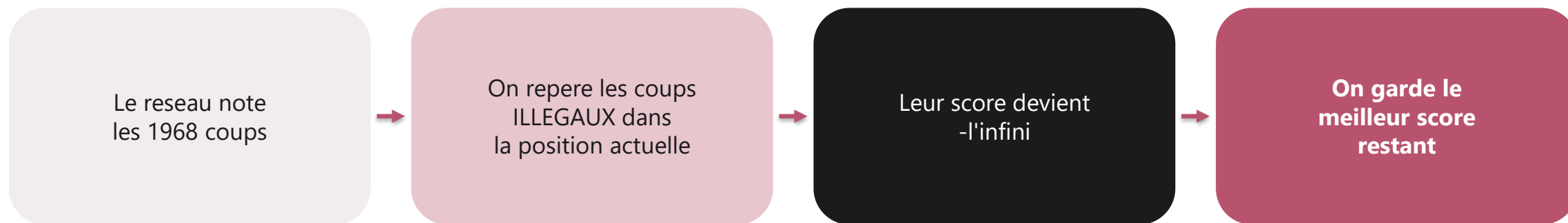
6 858 416
parametres au total

Pre-normalisation
stabilise l'entrainement d'un reseau
profond

Attention
relie directement 2 cases eloignees -
utile pour une tour ou un fou

Choisir un coup legal

Le bot ne peut pas tricher : c'est impossible par construction

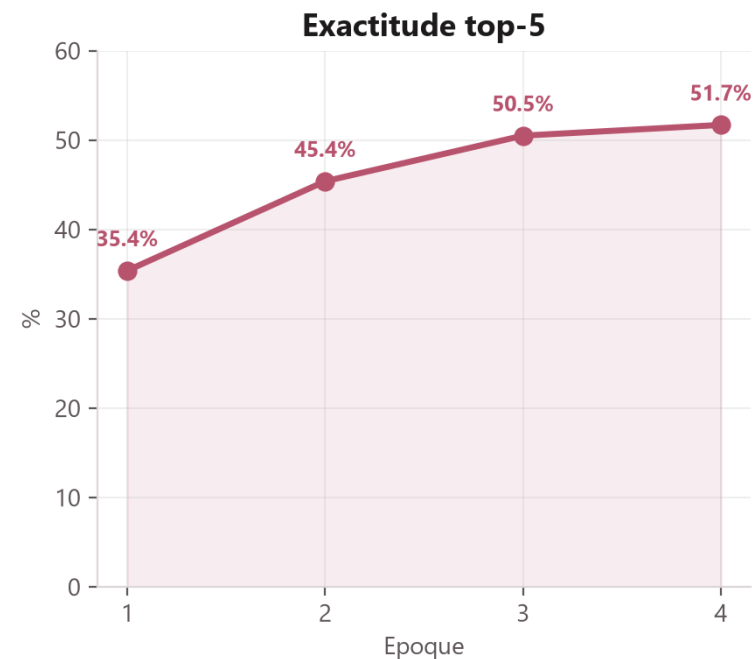
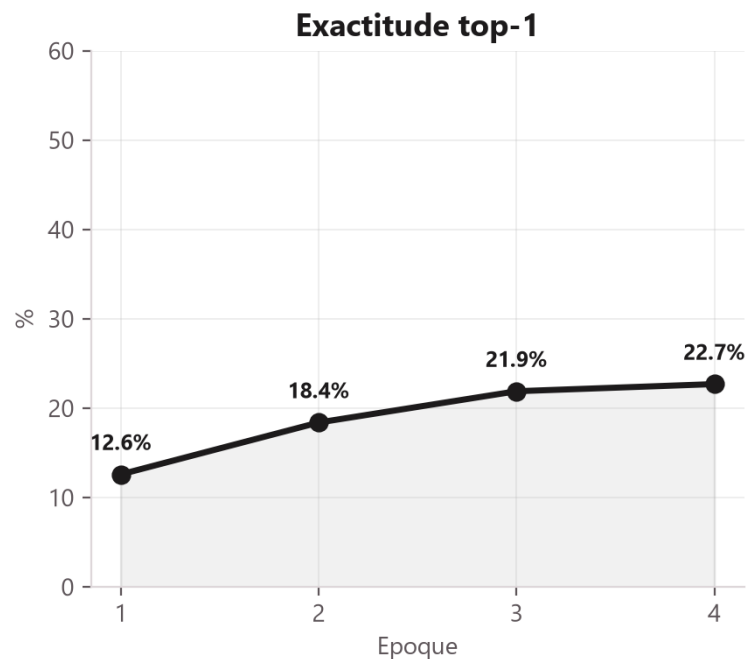
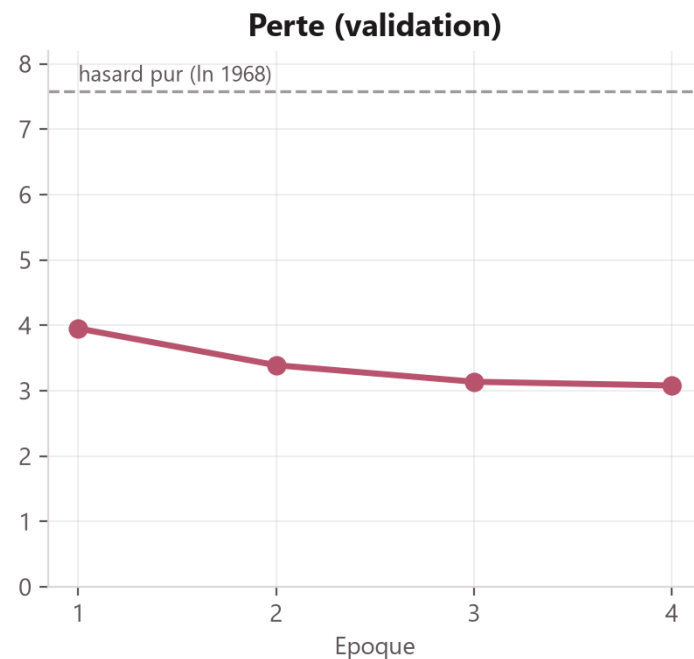


Un score de moins l'infini ne peut jamais etre le maximum : le coup illegal ne peut donc jamais etre choisi, quelle que soit la position.

ENTRAINEMENT

L'entrainement

4 epoques sur 1 million de positions, GPU T4 gratuit (Google Colab)



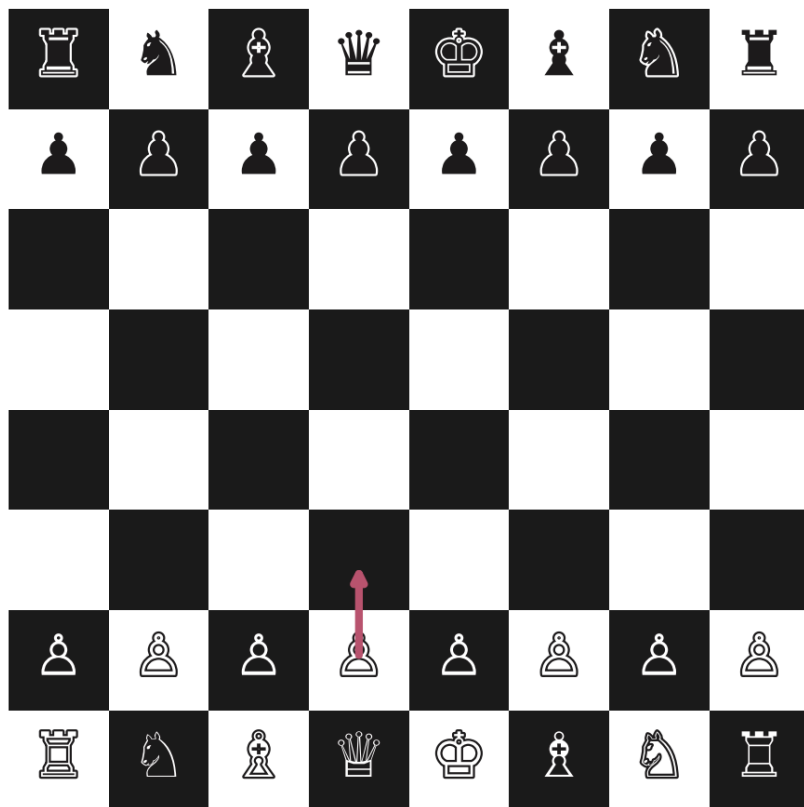
La perte descend regulierement sous le niveau du hasard (ligne pointillee), sans surapprentissage : les courbes d'entrainement et de validation restent proches.

RESULTATS

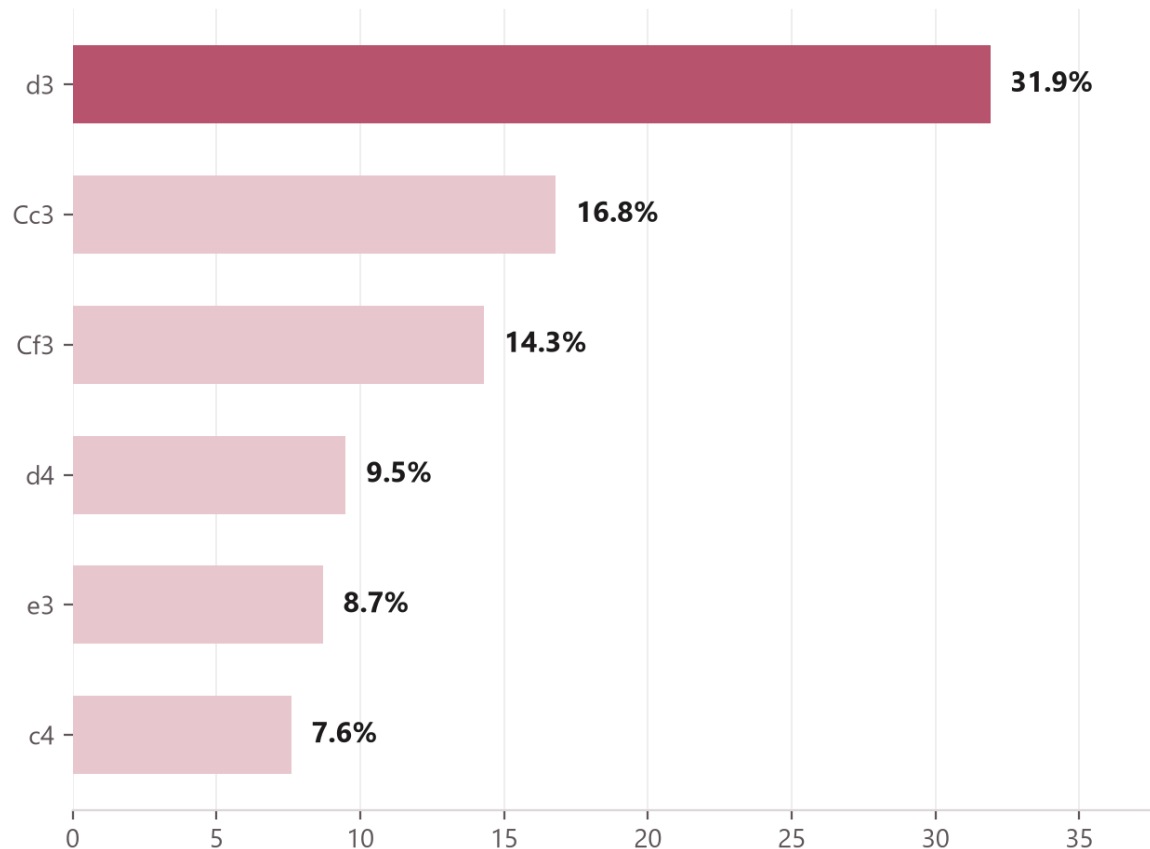
Il a appris les ouvertures

Sans qu'aucune regle ne lui ait ete enseignee

Position de depart



Coups envisages par le Transformer



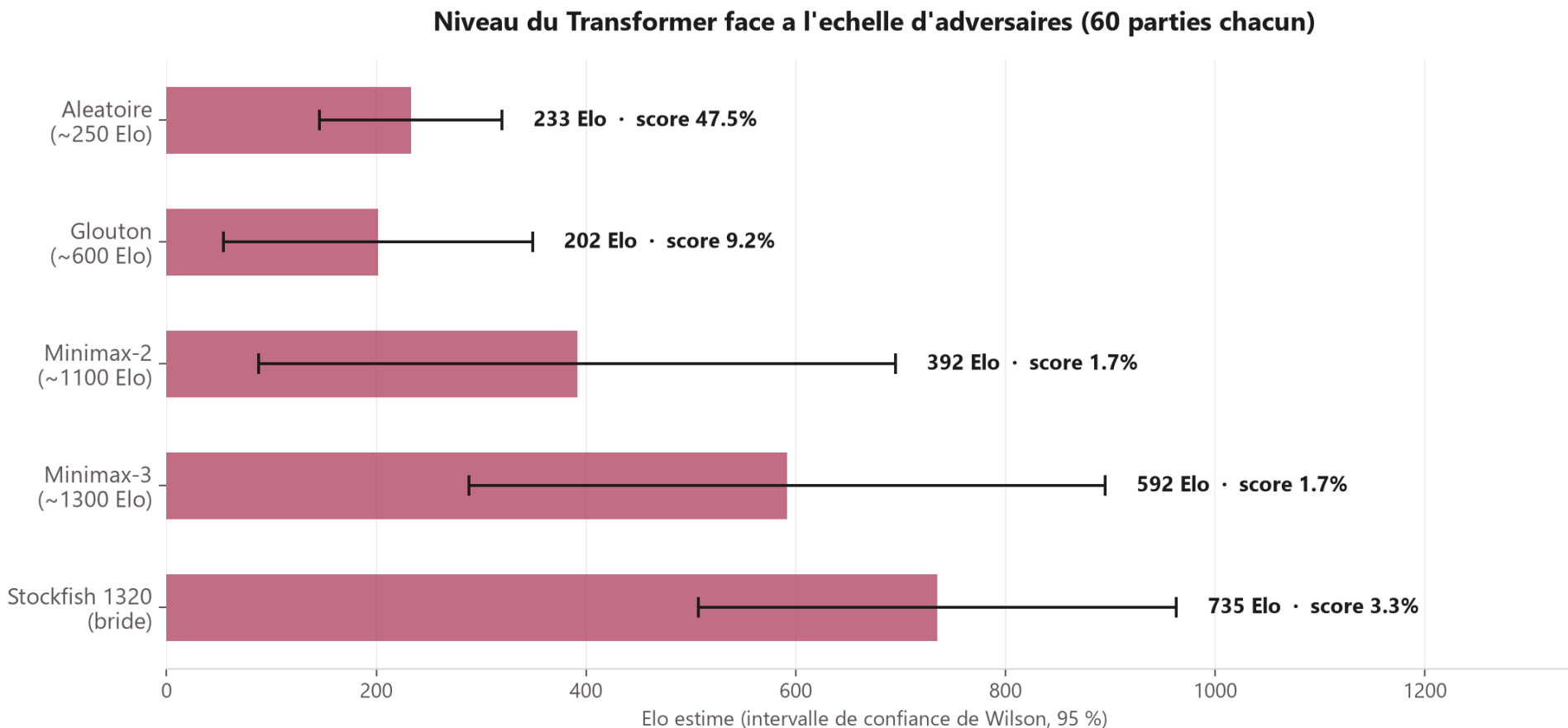
Probabilite donnee par le reseau

Dans la position de depart, le reseau concentre ses propositions sur le developpement des cavaliers et l'occupation du centre : des principes d'ouverture reels.

RESULTATS

Le niveau mesure en parties reelles

60 parties par adversaire, couleurs alternees, intervalles de confiance de Wilson (95 %)



Face a Stockfish bride a son niveau minimum (1320), le bot perd toutes ses parties : son niveau reel est nettement en dessous.

Analyse : ce que ca revele

- ▶ **Force : la connaissance des principes**

developpement, occupation du centre, acquis par simple observation, sans recherche.

- ▶ **Faiblesse : un jeu passif**

51 nulles sur 60 parties contre l'aleatoire. Le bot atteint des positions gagnantes mais ne les convertit pas, il tourne en rond.

- ▶ **Interpretation**

sans recherche, le modele ne verifie jamais une suite de coups. Il reconnait des schemas, mais ne calcule pas les consequences. Un entrainement plus long (les courbes montaient encore a la 4e epoque) aurait probablement ameliore ce point.

LIVRABLE

L'application

Jouer contre le bot, et voir ce qu'il pense

- ▶ **Mode Jouer**

on affronte le Transformer (ou les adversaires de reference) dans le navigateur.

- ▶ **Mode spectateur**

deux bots s'affrontent automatiquement, on regarde et on commente.

- ▶ **Le detail cle**

a chaque coup, les probabilites reellement sorties du reseau s'affichent : on voit litteralement le modele reflechir.

Limites et perspectives

- ▶ **Passer a l'action-value**

predire la probabilite de gagner de chaque coup, comme DeepMind, plutot que d'imiter le coup humain.

- ▶ **Annoter avec Stockfish**

apprendre du MEILLEUR coup calcule par un moteur, pas seulement du coup joue par un humain.

- ▶ **Ajouter une recherche legere**

une profondeur 2-3 guidee par les propositions du reseau corrigerait probablement l'essentiel des erreurs tactiques.

- ▶ **Entrainer plus longtemps**

les courbes etaient encore croissantes ; un GPU stable (non sujet aux deconnexions) permettrait d'aller plus loin.

Gestion de projet

Un binome, un depot Git versionne

► Naadjath & Rajaa

travail mixte sur l'ensemble de la chaine plutot qu'une repartition stricte par axe : donnees, modele, evaluation et application ont ete avancees en binome.

► Suivi

Git/GitHub avec des commits reguliers des deux membres, ce qui retrace la progression reelle du projet.

► Difficultes reelles rencontrees

lecture des donnees initialement trop lente (corrigee, x18) · Elo infini a 100 % de victoires (corrige par la methode de Wilson) · deconnexions de l'environnement Colab gratuit (contournees par sauvegarde sur Drive a chaque epoque).

Conclusion

- ▶ **Un Transformer apprend a jouer sans recherche**

juste en observant des parties, par clonage comportemental.

- ▶ **Niveau modeste (~250 Elo) mais reel**

principes d'ouverture correctement acquis, jeu de fin de partie encore faible.

- ▶ **Une demarche complete et rigoureuse**

chaine reproductible, encodage teste, evaluation avec intervalles de confiance.

- ▶ **Le point cle a retenir**

la difference entre connaissance (ce que le reseau a memorise) et calcul (la recherche qu'on a volontairement retiree).

Merci

Questions ?